Completing 4D Snake with Masked PPO and a Hamiltonian Reverse Curriculum

Author: Libor Burian

Repository: https://github.com/BurnyCoder/4d-snake-reinforcement-learning-agent - all numbers below come from the run artifacts copied to `reports/data/` and the per-experiment write-ups in `reports/experiments/`. This is an exploratory report of fixed checkpoints, each trained with seed 0. The scientific audit records the artifact checks, methodological defects and source-provenance limits; no new training or policy evaluation was run for these corrections.

Abstract

We build an N-dimensional snake game (default 4D, 4^4 = 256 cells, 8 moves) whose rules are implemented once as vectorised numpy over a body-age grid, so thousands of boards step together in a single process (126k-235k environment steps per second, 33k PPO steps per second end to end on a laptop GPU). We train sb3-contrib MaskablePPO agents to *complete* the game - fill every cell - with a legal-action mask from the environment, a sparse reward (+1 food, -1 death, +10 win, -0.001 per step) and a Backplay-style reverse curriculum whose demonstration is a Hamiltonian route (a cycle for even sizes; for odd sizes a cycle over all cells but one corner with a guarded detour). Scripted route following completed all evaluated 2^4, 3^4 and 4^4 episodes, providing a step-count reference (66 / 1,875 / 16,448 steps). On 2^4 the plain PPO checkpoints scored 0.88-0.93 completion; the nominally strict curriculum checkpoint scored 0.963 at the 5M-step budget while completing in 43.5 steps, a third fewer than the route follower's 65.9 on successful episodes. The curriculum gate mixes episodes begun at different frontiers, so its advances do not establish mastery of each stage. The strict arm also changed multiple curriculum parameters, and there are no independent training replications. On the larger boards the evaluated checkpoints recorded no completions within our budgets: on 3^4 (odd parity) all arms scored 54-57 % fill, and on 4^4 a 100M-step curriculum run moved the start frontier from 255 to 199 cells before its curriculum success collapsed; its selected checkpoint reached 40 % fill from the true start (best true-start training episode 66 %, `episodes.json`) and zero completions. Schedule decay and transfer from curriculum starts are hypotheses, not isolated causes of collapse; gate contamination is an additional confound. Behaviour-cloning the same network on 262k route-follower decisions spread over all fill levels (about twelve seconds of supervised training) yields a neural policy that completes the full 4^4 board in 100 % of deterministic evaluation episodes (3 evaluation seeds x 100), at the follower's 16,448 steps. The recorded PPO fine-tune did not establish an efficiency improvement and completed only one training episode, a failure. These findings describe the evaluated checkpoints, not algorithm-level superiority or guaranteed completion.

1. Introduction

Snake is a coverage problem in disguise: to win, the body must at the end form a Hamiltonian path of the grid, and every intermediate state must keep such a path reachable. In four dimensions the grid graph has degree 8, the intuition of "corners" disappears, and the author of the projected 4D snake eugeneko/Snake4D reports being unable to play it beyond a certain snake length. This project asks whether a model-free policy can learn to complete the 4D board, and builds the full pipeline needed to answer it: game, human play, scripted baselines, batched training, evaluation protocol and reports.

2. Related work

- **Masked PPO for snake.** linyiLYi/snake-ai trains MaskablePPO on a 12x12 board (its default `board_size`; its README does not report board completion), with a body gradient in the observation, and decaying learning rate and clip range; we adopt the masking, the body-age channel and the schedules, but not its distance-based progress bonus.
- **Invalid-action masking.** Huang and Ontanon (2020) justify masking theoretically and show its value in settings with many invalid actions. We use masking but did not ablate it, so its necessity in this game is not established.
- **Reverse curricula.** Backplay (Resnick et al. 2018) starts near the end of a demonstration and moves the start backwards; Salimans and Chen (2018) gate the move on the success rate. Our demonstration is a Hamiltonian route rather than a human trajectory.
- **Hamiltonian-cycle agents.** twanvl/snake benchmarks cycle followers and perturbed cycles (0 % losses); the perturbed-cycle rule "any shortcut must result in the head not overtaking the tail" is from johnflux's

Nokia-snake write-up (2015). A 2D grid graph with both sides $>= 2$ is Hamiltonian when one side is even (Skiena 1990, via MathWorld), and an even cell count is necessary because grid graphs are bipartite (Itai, Papadimitriou and Szwarcfiter 1982, who also show the Hamilton-cycle problem NP-complete for general grid graphs); `hamilton.ham_cycle` lifts a 2D cycle to d dimensions and verifies the result, and the odd-board cycle over all cells but a corner is our own construction, also verified.

- **Imitation.** Behaviour cloning (Pomerleau 1988) learns the expert's action from the observation; its covariate-shift failure mode and the DAgger remedy are due to Ross, Gordon and Bagnell (2011).
- **AlphaSnake** (Du et al. 2022) is, by its authors' account, the first published learned agent with a >50 % win rate on a 10x10 board, using MCTS; its Table 1 compares it with a Hamiltonian-cycle strategy among other baselines, which we also use as a scripted baseline.
- **Shaping.** Ng, Harada and Russell (1999) give policy-invariance conditions for potential-based reward shaping; episodic boundary conditions also matter (Grzes 2017). Shaping is available but off in the recorded experiments.
- **Empirical design.** Patterson et al. (2024) distinguish experimental units and sources of variability in RL comparisons. Our multiple evaluation seeds quantify fixed-checkpoint variation; they do not replace independent training seeds.

3. The game

State: a body-age grid `age[cell]` (0 empty, 1 tail, `length` head), head and food indices. A step resolves the target cell, decrements every age unless the snake eats (so the tail vacates first and following the tail is legal), tests occupancy, writes the head, checks `length == C` before spawning food uniformly over free cells. Starvation (more than `4C` consecutive idle steps) and an absolute cap (`C^2` steps) truncate the episode; the truncating step is paid `r_step` like any other and there is no death penalty. Observation: `4C + 2` floats (body, time-to-vacate `age/length`, head one-hot, food one-hot, `length/C`, hunger). Mask: inside the board, free after the tail moves, not the neck; an all-false row falls back to in-bounds moves because MaskablePPO masks with `-1e8`, not `-inf`. Rendering folds the 4D board into a z-by-w grid of x-by-y tiles.

Parity: the grid graph is bipartite, so a Hamiltonian cycle needs an even cell count. 4^4 has one (constructed recursively and verified); 3^4 (41/40 colour split) has none, and completing it requires the final body to be a Hamiltonian path with both ends in the larger colour class.

4. Method

Agent. MaskablePPO, MLP [512, 512] for policy and value, gamma 0.99, GAE 0.95, learning rate 3e-4 -> 1e-5 and clip range 0.2 -> 0.05 (both linear), entropy 0.01, target KL 0.03, 4 epochs, minibatch 8192, 4096 batched environments with 64-step rollouts (262,144 samples per update). Defaults were chosen by the throughput benchmark (exp00) and the reference agent's schedules.

Batched environment. One `SnakeBatch` holds all boards; a custom SB3 `VecEnv` exposes it and `VecMonitor` records episodes. The benchmark measured 10x higher PPO throughput than `DummyVecEnv/SubprocVecEnv` over single environments and showed the PPO update, not the environment, to be the bottleneck (minibatch 2048 -> 8192 lifts throughput 19k -> 31k fps on the best row; `reports/data/exp00_benchmark/benchmark.json`).

Curriculum. With the current defaults, 80 % of resets place the snake as a segment of length `U[hi - 4, hi]` along a random rotation of the Hamiltonian route, starting at `hi = C - 1`; the frontier moves back by `max(1, C/64)` cells when at least 500 accumulated curriculum episodes exceed 90 % success; 20 % of resets start from length 1 and evaluation always starts from length 1. Per-run configurations override these defaults. The historical callback admits episodes begun before a frontier change into the next gate's statistic. Its logged success rate is therefore a mixture of stages, not a valid current-stage mastery measure. Reset proportions also differ from transition proportions because episode durations differ; see the audit for reconstructed examples.

Baselines. The route follower (Hamiltonian cycle, or cycle-minus-corner with a detour that skips at most as many cycle cells as are free) and uniformly random legal play.

Evaluation. 100 episodes x 3 seeds, deterministic and stochastic, masked `evaluate_policy`; metrics: completion rate, mean fill, steps to complete conditional on winning, win-within-K. Each learned checkpoint has one training seed (0); +− values below are standard deviations across evaluation-seed means, not confidence intervals or training-seed variability. Checkpoints were selected using in-training evaluation. The evaluation protocol states the uncertainty interpretation and selection limitations.

5. Experiments

Experiment configurations live in `experiments/*.env`; every write-up follows question -> hypothesis -> setup -> results -> learnings. This organization does not establish preregistration: some configurations were committed after their runs, and recorded commit IDs do not always identify the complete executed source. These analyses are retrospective.

- exp00 - throughput benchmark: batched env on CUDA, 4096 envs; minibatch 8192 gives 31k fps (33k at 16384; 8192 adopted for more gradient steps per rollout).
- exp01 - scripted baselines: route follower 100 % on 2^4 / 3^4 / 4^4 (65.9 / 1,874.5 / 16,447.8 steps); random legal play 29 % / 0 % / 0 % (`reports/data/exp01_*/summary.json`). The first odd-board follower (an open Gray-code path) reached about 4 % fill (run not archived, see exp01) and was replaced by the cycle-minus-corner construction.
- exp02 - 2^4: plain PPO 0.877 (5M) and 0.933 (20M) deterministic completion; loosely gated Backplay (rho 0.2) 0.850; strictly gated Backplay (rho 0.9, window 4) 0.963 at 5M with 43.5 steps per completion (`reports/data/exp02*_best/summary.json`). The strict checkpoint's failed episodes included endings with one cell left. Relative to the loose arm, the strict arm changed gate threshold, window width and minimum episode count together, so the score difference cannot be attributed to the threshold alone. Its settings became the defaults.
- exp03 - 3^4 (81 cells, odd): plain PPO, strict Backplay (gate 0.9) and relaxed Backplay (gate 0.8) at 30M steps each all end at 0.54-0.57 fill and 0 completions; the curriculum frontier never left the endgame (80 -> 76-78); these records do not isolate failure to learn the corner detour (`reports/experiments/exp03_ppo_3x4.md`).
- exp04 - 4^4 (256 cells): Backplay (gate 0.8, window 16, step 8) for 100M steps / 54 min: seven frontier advances (255 -> 199) between 18.6M and 53.7M steps, none in the remaining 46M, and the logged curriculum success rate fell below 0.001 from about 73M steps and to exactly zero from about 77M steps as the learning rate and clip range decayed towards their end values; true-start fill 0.40 (best true-start training episode 0.66), completion 0.000 (`reports/experiments/exp04_ppo_4x4.md`, `reports/data/exp04_ppo_4x4*/`).
- exp05 - 4^4 imitation warm start: behaviour cloning of the route follower records 1.000 expert-action accuracy on training minibatches, without a held-out accuracy measurement, and the cloned network completes the board in 1.000 +- 0.000 of deterministic episodes (0.787 +- 0.041 when sampling), 16,448 steps per game. exp05b fine-tuned it with PPO for 20M steps (lr 1e-4, clip 0.1, entropy coefficient 0, no curriculum): completion stayed at 1.00 in all nine in-training evaluations, and the selected checkpoint's completion-step mean remained close to the clone's. `approx_kl` stayed below 2e-5 but was nonzero, as was policy entropy: this does not establish an unchanged policy or identical sampled and argmax actions. The run completed only one training episode, a failure, limiting evidence about learning from successful full trajectories. No efficiency improvement or causal explanation was established (`reports/experiments/exp05_bc_4x4.md`, `reports/data/exp05*`).

The generated cross-run table is `reports/all_experiments.md`; `reports/networks.md` puts every trained network beside the two scripted baselines in plain words (completion, fill, steps, and steps per food against a geometric floor).

6. Discussion

On the 16-cell board, the checkpoint labelled strict Backplay had the highest observed completion among the tested from-scratch checkpoints. This is a descriptive ranking: one training seed per arm, multiple simultaneous curriculum changes and contaminated stage gates prevent an algorithm-level or threshold-specific conclusion. Some of the strict checkpoint's failures ended with one free cell, but that outcome alone does not identify its internal planning mechanism.

On 81 and 256 cells the evaluated from-scratch checkpoints recorded no completions. Possible explanations include limited transfer between curriculum and true-start distributions and learning-rate/clip-range decay tied to the total budget. Neither was isolated by an ablation; low true-start performance does not prove that curriculum states are unreachable. The curriculum defect is a further explanation: stage advances can be supported by results from older, easier stages. No causal decomposition is possible from these runs. The odd board has a parity constraint and the scripted baseline uses a corner detour, but the learned policies' failure mechanism is not established.

The imitation checkpoint demonstrates neural-policy completion in the archived deterministic 4^4 evaluation. Its training accuracy measures fit to expert minibatches; matching the follower's completion score and mean duration does not establish a head-only internal rule or equivalence on every state. Sampling produced failures, and the PPO fine-tune provides little evidence about successful full-trajectory learning. Efficiency and sampling robustness remain research targets. Frontier-driven schedules, more complete training trajectories, and decision-time search are candidates for future controlled experiments, not validated remedies.

The audit and evaluation protocol give the complete limitations: conditional fixed-checkpoint uncertainty, incomplete historical source provenance, retrospective hypotheses and missing independent training replications. No demonstrated direct train/test trajectory leakage was found; this does not establish the absence of all selection bias. Historical timeout-target aliasing affected some runs, but the archived 4^4 curriculum run had no timeouts and that defect cannot explain its collapse. The measured scores remain descriptive evidence; neither statistical significance nor causal identification can be supplied by editing the reports.

7. Conclusion

The repository delivers a playable 4D snake, a batched training stack measured at 33k PPO steps per second on one laptop GPU, scripted route baselines and archived experiment artifacts. The selected strict-curriculum MaskablePPO checkpoint completed the 16-cell board in 96 % of its evaluation episodes; the evaluated from-scratch checkpoints recorded no 3^4 or 4^4 completions within the 30M- and 100M-step budgets. The behaviour-cloned 4^4 checkpoint completed 100 % of its archived deterministic episodes. These observations survive arithmetic verification, while stage-mastery, algorithm superiority and causal mechanism claims do not. Whether PPO can make the clone more efficient remains unresolved.

References

- Patterson, A., Neumann, S., White, M. and White, A. (2024). Empirical Design in Reinforcement Learning. JMLR 25(318), 1-63. https://jmlr.org/papers/v25/23-0183.html
- Huang, S. and Ontanon, S. (2020). A Closer Look at Invalid Action Masking in Policy Gradient Algorithms. arXiv:2006.14171.
- Resnick, C. et al. (2018). Backplay: "Man muss immer umkehren". arXiv:1807.06919.
- Salimans, T. and Chen, R. (2018). Learning Montezuma's Revenge from a Single Demonstration. arXiv:1812.03381.
- Ng, A., Harada, D. and Russell, S. (1999). Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. ICML, 278-287. https://dl.acm.org/doi/10.5555/645528.657613
- Grzes, M. (2017). Reward Shaping in Episodic Reinforcement Learning. AAMAS. https://dl.acm.org/doi/10.5555/3091125.3091208
- Itai, A., Papadimitriou, C. H. and Szwarcfiter, J. L. (1982). Hamilton Paths in Grid Graphs. SIAM J. Comput. 11(4), 676-686. https://doi.org/10.1137/0211056
- Skiena, S. (1990). Implementing Discrete Mathematics: Combinatorics and Graph Theory with Mathematica. Addison-Wesley, p. 148 (as cited by MathWorld, https://mathworld.wolfram.com/GridGraph.html).
- Du, Y. et al. (2022). AlphaSnake: Policy Iteration on a Nondeterministic NP-hard Markov Decision Process. arXiv:2211.09622.
- Andrychowicz, M. et al. (2020). What Matters in On-Policy Reinforcement Learning? arXiv:2006.05990.
- Raffin, A. et al. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. JMLR 22(268), 1-8. https://jmlr.org/papers/v22/20-1364.html
- Schulman, J. et al. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.
- Towers, M. et al. (2024). Gymnasium: A Standard Interface for Reinforcement Learning Environments. arXiv:2407.17032.
- Pomerleau, D. A. (1988). ALVINN: An Autonomous Land Vehicle in a Neural Network. NeurIPS. https://proceedings.neurips.cc/paper/1988/hash/812b4ba287f5ee0bc9d43bbf5bbe87fb-Abstract.html
- Ross, S., Gordon, G. and Bagnell, D. (2011). A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning (DAgger). AISTATS. arXiv:1011.0686.
- Müller, R., Kornblith, S. and Hinton, G. (2019). When Does Label Smoothing Help? NeurIPS. arXiv:1906.02629.
- Florensa, C. et al. (2017). Reverse Curriculum Generation for Reinforcement Learning. CoRL. arXiv:1707.05300.
- Silver, D. et al. (2017). Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. arXiv:1712.01815.
- Sutton, R. S. and Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press. http://incompleteideas.net/book/the-book-2nd.html
- johnflux (2015). Nokia 6110 Part 3 - Algorithms. https://johnflux.com/2015/05/02/nokia-6110-part-3-algorithms/
- linyiLYi/snake-ai, twanvl/snake, instadeepai/jumanji, Pella86/Snake4d (see THIRD_PARTY_NOTICES.md).

All experiments

Generated by `uv run snake4d report` from the artifacts in `runs/` (small copies in `reports/data/`). Each experiment's question, hypothesis, analysis and learnings are in its own file:

- exp00_benchmark
- exp01_baselines
- exp02_ppo_2x4
- exp03_ppo_3x4
- exp04_ppo_4x4
- exp05_bc_4x4

Metrics by run

Scores describe the archived checkpoints. The +- values are variation across evaluation seeds, not training replications or confidence intervals. See the scientific audit for interpretation and provenance limits.

exp01_route_2x4

metric	value
phase	evaluate
board	2^4
success_det	1.000 +- 0.000
fill_det	1
steps_to_complete_det	65.8667
run_dir	20260904-192818_evaluate_exp01_route_2x4

exp01_random_2x4

metric	value
phase	evaluate
board	2^4
success_det	0.293 +- 0.048
fill_det	0.816
steps_to_complete_det	129.352
run_dir	20260904-192823_evaluate_exp01_random_2x4

exp01_random_3x4

metric	value
phase	evaluate
board	3^4
success_det	0.000 +- 0.000
fill_det	0.226
run_dir	20260904-192835_evaluate_exp01_random_3x4

exp01_route_4x4

metric	value
phase	evaluate
board	4^4
success_det	1.000 +- 0.000
fill_det	1
steps_to_complete_det	16447.8
run_dir	20260904-192857_evaluate_exp01_route_4x4

exp02_ppo_2x4

metric	value
phase	train
board	2^4
timesteps	5,013,504
wall_min	2.167
fps	38,526

fill_final	0.98
fill_true_start_final	0.98
eval_success_final	0.88
eval_success_best	0.91
eval_success_best_at	2,949,120
run_dir	20260904-192913_train_exp02_ppo_2x4

exp01_route_3x4

metric	value
phase	evaluate
board	3^4
success_det	1.000 +- 0.000
fill_det	1
steps_to_complete_det	1874.5
run_dir	20260904-194950_evaluate_exp01_route_3x4

exp02b_ppo_2x4_backplay

metric	value
phase	train
board	2^4
timesteps	5,013,504
wall_min	2.3
fps	36,208
fill_final	0.983
fill_true_start_final	0.983
eval_success_final	0.9
eval_success_best	0.91
eval_success_best_at	3,604,480
run_dir	20260904-194950_train_exp02b_ppo_2x4_backplay

exp01_random_4x4

metric	value
phase	evaluate
board	4^4
success_det	0.000 +- 0.000
fill_det	0.077
run_dir	20260904-195012_evaluate_exp01_random_4x4

exp02c_ppo_2x4_long

metric	value
phase	train
board	2^4
timesteps	20,021,248
wall_min	8.567
fps	38,901
fill_final	0.986
fill_true_start_final	0.986
eval_success_final	0.87
eval_success_best	0.96
eval_success_best_at	17,039,360
run_dir	20260904-195216_train_exp02c_ppo_2x4_long

exp02d_ppo_2x4_backplay_strict

metric	value
phase	train
board	2^4
timesteps	5,013,504
wall_min	2.117
fps	39,204
fill_final	0.994
fill_true_start_final	0.994
eval_success_final	0.95
eval_success_best	1

eval_success_best_at	3,604,480
run_dir	20260904-204019_train_exp02d_ppo_2x4_backplay_strict

exp03a_ppo_3x4_nocur

metric	value
phase	train
board	3^4
timesteps	30,015,488
wall_min	9.9
fps	50,518
fill_final	0.548
fill_true_start_final	0.548
eval_success_final	0
eval_success_best	0
eval_success_best_at	1,310,720
run_dir	20260904-204245_train_exp03a_ppo_3x4_nocur

exp02_ppo_2x4_best

metric	value
phase	evaluate
board	2^4
success_det	0.877 +- 0.041
success_stoch	0.853 +- 0.034
fill_det	0.987
steps_to_complete_det	34.4221
run_dir	20260904-204250_evaluate_exp02_ppo_2x4_best

exp02b_ppo_2x4_backplay_best

metric	value
phase	evaluate
board	2^4
success_det	0.850 +- 0.029
success_stoch	0.843 +- 0.026
fill_det	0.98
steps_to_complete_det	34.5647
run_dir	20260904-204300_evaluate_exp02b_ppo_2x4_backplay_best

exp02c_ppo_2x4_long_best

metric	value
phase	evaluate
board	2^4
success_det	0.933 +- 0.005
success_stoch	0.863 +- 0.025
fill_det	0.991
steps_to_complete_det	35.6679
run_dir	20260904-204310_evaluate_exp02c_ppo_2x4_long_best

exp02d_ppo_2x4_backplay_strict_best

metric	value
phase	evaluate
board	2^4
success_det	0.963 +- 0.009
success_stoch	0.960 +- 0.033
fill_det	0.994
steps_to_complete_det	43.4983
run_dir	20260904-204320_evaluate_exp02d_ppo_2x4_backplay_strict_best

exp03b_ppo_3x4_backplay

metric	value
phase	train
board	3^4
timesteps	30,015,488

wall_min	9.7
fps	51,486
fill_final	0.909
fill_true_start_final	0.532
eval_success_final	0
eval_success_best	0
eval_success_best_at	1,310,720
run_dir	20260904-205247_train_exp03b_ppo_3x4_backplay

exp03a_ppo_3x4_nocur_best

metric	value
phase	evaluate
board	3^4
success_det	0.000 +- 0.000
success_stoch	0.000 +- 0.000
fill_det	0.566
run_dir	20260904-210002_evaluate_exp03a_ppo_3x4_nocur_best

exp04_ppo_4x4

metric	value
phase	train
board	4^4
timesteps	100,139,008
wall_min	53.667
fps	31,096
fill_final	0.708
fill_true_start_final	0.4
eval_success_final	0
eval_success_best	0
eval_success_best_at	2,097,152
run_dir	20260904-210249_train_exp04_ppo_4x4

exp03c_ppo_3x4_backplay_relaxed

metric	value
phase	train
board	3^4
timesteps	30,015,488
wall_min	10.633
fps	47,033
fill_final	0.898
fill_true_start_final	0.536
eval_success_final	0
eval_success_best	0
eval_success_best_at	1,310,720
run_dir	20260904-215712_train_exp03c_ppo_3x4_backplay_relaxed

exp03b_ppo_3x4_backplay_best

metric	value
phase	evaluate
board	3^4
success_det	0.000 +- 0.000
success_stoch	0.000 +- 0.000
fill_det	0.549
run_dir	20260904-221102_evaluate_exp03b_ppo_3x4_backplay_best

exp03c_ppo_3x4_backplay_relaxed_best

metric	value
phase	evaluate
board	3^4
success_det	0.000 +- 0.000
success_stoch	0.000 +- 0.000
fill_det	0.542
run_dir	20260904-221117_evaluate_exp03c_ppo_3x4_backplay_relaxed_best

exp04_ppo_4x4_best

metric	value
phase	evaluate
board	4^4
success_det	0.000 +- 0.000
success_stoch	0.000 +- 0.000
fill_det	0.401
run_dir	20260904-221133_evaluate_exp04_ppo_4x4_best

exp05_bc_4x4

metric	value
phase	imitate
board	4^4
run_dir	20260904-221730_imitate_exp05_bc_4x4

exp05_bc_4x4_eval

metric	value
phase	evaluate
board	4^4
success_det	1.000 +- 0.000
success_stoch	0.787 +- 0.041
fill_det	1
steps_to_complete_det	16447.8
run_dir	20260904-221751_evaluate_exp05_bc_4x4_eval

exp05b_ppo_4x4_from_bc

metric	value
phase	train
board	4^4
timesteps	20,185,088
wall_min	22.517
fps	14,930
fill_final	0.191
fill_true_start_final	0.191
eval_success_final	1
eval_success_best	1
eval_success_best_at	2,097,152
run_dir	20260904-221940_train_exp05b_ppo_4x4_from_bc

exp05b_ppo_4x4_from_bc_best

metric	value
phase	evaluate
board	4^4
success_det	1.000 +- 0.000
success_stoch	0.763 +- 0.017
fill_det	1
steps_to_complete_det	16413.6
run_dir	20260904-224248_evaluate_exp05b_ppo_4x4_from_bc_best

exp00_benchmark

metric	value
phase	bench
board	4^4
run_dir	20260905-010410_bench_exp00_benchmark

Figures: `reports/figures/<run>_curves.png` and `<run>_fill_hist.png` per training run.

The ten trained networks beside the two scripted baselines

(README.md has the overview and the headline results; docs/game_rules.md, docs/rl_design.md and docs/evaluation.md define the game, the learning setup and the metrics; experiments/ analyses each experiment. This page only puts the ten trained networks side by side with the two scripted baselines, in plain words, and repeats none of those texts.)

All scores are the `deterministic=True` entries of `data/<run>/summary.json` (100 episodes x 3 seeds from length 1, protocol in docs/evaluation.md; the sampling-mode numbers are in the experiment write-ups and in the `success_stoch` column of all_experiments.md). The baselines are `data/exp01_route_<n>x4`, the Hamiltonian route follower that completed every evaluated episode and supplies a step-count reference, and `data/exp01_random_<n>x4`, random legal play, the floor (exp01). Budgets, curricula and wall times come from the `experiments/*.env` files and the `wall_min` column of all_experiments.md. Steps per food, the geometric floor, the non-eating share and the weight counts are the authors' own arithmetic from those files, an estimate of route efficiency rather than a measurement. `tests/test_docs_numbers.py` recomputes every bold number, every baseline number and every verdict on this page from `reports/data/`, and checks the weight counts against the built model.

All ten networks use training seed 0. The three evaluation seeds sample outcomes for each fixed checkpoint; they are not independent training replications. The scores remain descriptive observations, and do not establish algorithm-level superiority or training robustness. See evaluation uncertainty and the scientific audit for confidence intervals, the stale-frontier defect, and source-provenance limits. Hypotheses in the experiment reports are retrospective; independent preregistration is not established.

The weights are downloadable: every network on this page has a Hugging Face Hub repo named `BurnyCoder/4d-snake-<run>` (the evaluated checkpoint, `config.json`, `versions.json`, the evaluation files, a model card and, for training runs, the SB3 log and figures), grouped in the collection 4D Snake RL: all evaluated networks, uploaded by `snake4d publish` (docs/architecture.md); `snake4d watch` plays any of them in a window (the "Play and watch" section of README.md).

What all ten networks share

component	technical, where it is defined	in plain words
Domain	4-dimensional snake on an `n^4` hypercube with `n` = 2, 3 or 4 cells per side, so 16, 81 or 256 cells and 8 moves, plus or minus along each axis: docs/game_rules.md.	Ordinary snake with a 4D cube for a board. You cannot picture it, but the rules are the usual ones: eat, grow, never hit a wall or yourself.
Input	A flat vector of `4C + 2` numbers: body map, time-to-vacate map, head one-hot, food one-hot, length and hunger (the observation table of docs/game_rules.md).	The network never sees a picture. It gets one spreadsheet row describing the whole board, 66 numbers on the smallest board and 1,026 on the largest.
Architecture	sb3-contrib `MaskablePPO` with `MlpPolicy` and `net_arch {"pi": [512, 512], "vf": [512, 512]}` (train.py; https://sb3-contrib.readthedocs.io/en/master/modules/ppo_mask.html); 8 move scores and 1 value out; 0.60M / 0.86M / 1.58M weights on 2^4 / 3^4 / 4^4, own arithmetic from the layer sizes, checked against the built model in `tests/test_docs_numbers.py`.	The plainest kind of neural net: two layers of 512 units turn the input row into a score for each of the 8 moves, and a twin stack guesses how well the game is going. No convolution, no memory between steps.
Action masking	Moves into walls, occupied cells or the neck are removed before the choice; when no move is legal the mask falls back to every in-bounds move and the snake dies (the action-mask section of docs/game_rules.md, `physics.action_masks`; invalid-action masking: Huang and Ontanon 2020, https://arxiv.org/abs/2006.14171).	The game hands the net a list of legal moves, so it never takes an avoidable fatal move. It dies only when every move is fatal, after boxing itself in.

RL algorithm and reward	MaskablePPO on `+1` food, `−1` death, `+10` win, `−0.001` per step; discount 0.99, entropy bonus 0.01, linearly decaying learning rate and clip range (docs/rl_design.md; PPO: Schulman et al. 2017, https://arxiv.org/abs/1707.06347).	Trial and error across thousands of games played at once. Moves that led to more reward get more probability next time, the rest less. A small bonus for variety keeps it exploring.
Imitation algorithm	Behaviour cloning of the route follower: cross-entropy of the expert move under the masked policy, Adam 1e-3, 20 epochs over 262,144 samples (imitation.py, exp05; Pomerleau 1988, https://proceedings.neurips.cc/paper/1988/hash/812b4ba287f5ee0bc9d43bbf5bbe87fb-Abstract.html).	Flashcards. Show the board, reveal the teacher's move, adjust the weights until the net gives the same answer. The teacher is a fixed loop through every cell.
Training data	The simulator generates PPO experience live; BC stores a collected expert dataset in memory for repeated training epochs (the `fps` column of all_experiments.md; docs/benchmark.md, exp05).	The game supplies the experience; imitation revisits the same collected examples.
Curriculum	Backplay along the Hamiltonian route: each reset chooses a curriculum start with probability 0.8 near a frontier that begins one cell short of a full board (two short on the odd 3^4 board). The historical gate pooled completions from different reset frontiers, so an advance does not establish mastery of the current stage (audit, docs/rl_design.md; Backplay: Resnick et al. 2018, https://arxiv.org/abs/1807.06919; success-gated advance: Salimans and Chen 2018, https://arxiv.org/abs/1812.03381).	Practise near the end of the game, then move starts earlier when the logged gate passes. Reset probabilities do not equal fractions of training moves. Evaluation starts from length 1.
Baselines	Route follower (completed every evaluated episode; on the odd 3^4 board the cycle covers all cells but one corner and a timed detour takes the corner, docs/game_rules.md) and masked random play, evaluated by the same protocol: exp01. Cycle-following strategy: twanvl/snake, https://github.com/twanvl/snake; johnflux 2015, https://johnflux.com/2015/05/02/nokia-6110-part-3-algorithms/.	The route is the completion reference every network is measured against. Random chooses uniformly among masked legal moves.
Evaluation	100 episodes x 3 seeds through masked `evaluate_policy`, a deterministic and a sampling pass for trained models and one deterministic pass for the scripted baselines (docs/evaluation.md); this page shows the deterministic pass.	Every number below comes from 300 evaluation games, 100 per seed, with the network always taking its top-scored move.
Efficiency metrics	Steps per food, geometric floor (2.00 / 3.56 / 5.00 moves on 2^4 / 3^4 / 4^4), non-eating move share and won within `4C` moves: the derived-metrics section of docs/evaluation.md; the step-capped win rate follows AlphaSnake, whose Table 1 reports wins within 1,200 steps on a 10x10 board, https://arxiv.org/abs/2211.09622.	Steps per food is how far the snake walks between meals. Times floor compares this with the mean distance between two uniformly sampled board cells; it is a geometric reference, not the achievable shortest route during play. Non-eating moves is the share of moves that ate nothing.

The ten networks, scores beside baselines

How to read a score cell: the network's own score in bold, then the loop follower's score with a verdict, then random play's score with a verdict. Verdicts are from the network's point of view and are judged at the printed precision, so equal digits are a tie. These labels report arithmetic ordering only, not statistical significance, equivalence or a causal effect. `never` means no sampled evaluation episode finished; it does not imply a zero population success probability. Budgets are environment moves. Steps to finish condition on successful episodes.

run	board	training	completion	fill	steps to finish
exp02	2^4, 16 cells	MaskablePPO from scratch, no curriculum, 5M moves, 2.2 min (write-up)	**87.7 %** (loop 100.0 %, worse; random 29.3 %, better)	**0.987** (loop 1.000, worse; random 0.816, better)	**34.4** (loop 65.9, better; random 129.4, better)
exp02b	2^4	PPO, loose Backplay (advance at 20 % success, window 8), 5M moves, 2.3 min	**85.0 %** (loop 100.0 %, worse; random 29.3 %, better)	**0.980** (loop 1.000, worse; random 0.816, better)	**34.6** (loop 65.9, better; random 129.4, better)
exp02c	2^4	PPO, no curriculum, 20M moves, 8.6 min	**93.3 %** (loop 100.0 %, worse; random 29.3 %, better)	**0.991** (loop 1.000, worse; random 0.816, better)	**35.7** (loop 65.9, better; random 129.4, better)

exp02d	2^4	PPO, strict Backplay (advance at 90 % success, window 4), 5M moves, 2.1 min	**96.3 %** (loop 100.0 %, worse; random 29.3 %, better)	**0.994** (loop 1.000, worse; random 0.816, better)	**43.5** (loop 65.9, better; random 129.4, better)
exp03a	3^4, 81 cells	PPO, no curriculum, 30M moves, 9.9 min (write-up)	**0.0 %** (loop 100.0 %, worse; random 0.0 %, tie)	**0.566** (loop 1.000, worse; random 0.226, better)	never (loop 1,874.5; random never)
exp03b	3^4	PPO, strict Backplay (90 % gate, window 4), 30M moves, 9.7 min	**0.0 %** (loop 100.0 %, worse; random 0.0 %, tie)	**0.549** (loop 1.000, worse; random 0.226, better)	never (loop 1,874.5; random never)
exp03c	3^4	PPO, relaxed Backplay (80 % gate, window 8, frontier step 4), 30M moves, 10.6 min	**0.0 %** (loop 100.0 %, worse; random 0.0 %, tie)	**0.542** (loop 1.000, worse; random 0.226, better)	never (loop 1,874.5; random never)
exp04	4^4, 256 cells	PPO, Backplay (80 % gate, window 16, frontier step 8), 100M moves, 53.7 min (write-up)	**0.0 %** (loop 100.0 %, worse; random 0.0 %, tie)	**0.401** (loop 1.000, worse; random 0.077, better)	never (loop 16,447.8; random never)
exp05	4^4	behaviour cloning of the route follower, 262,144 samples x 20 epochs, about 12 s (write-up)	**100.0 %** (loop 100.0 %, tie; random 0.0 %, better)	**1.000** (loop 1.000, tie; random 0.077, better)	**16,447.8** (loop 16,447.8, tie; random never, better)
exp05b	4^4	PPO fine-tune from the exp05 clone, no curriculum, no entropy bonus, lr 1e-4 to 1e-5, 20M moves, 22.5 min	**100.0 %** (loop 100.0 %, tie; random 0.0 %, better)	**1.000** (loop 1.000, tie; random 0.077, better)	**16,413.6** (loop 16,447.8, better; random never, better)

What happened, one line per network (details and figures in the write-ups):

- exp02: climbs quickly and then plateaus (write-up); 23 of its 37 lost argmax games end with one cell left, the snake boxed in (`data/exp02_ppo_2x4_best/eval_episodes.csv`).
- exp02b: the loose gate ran the curriculum frontier from 15 to 1 in 15 seconds of training (`data/exp02b_ppo_2x4_backplay/run.log`). Its observed completion was below plain PPO's; neither absence of learning nor equivalence is established (write-up).
- exp02c: the checkpoints differ from 87.7 % to 93.3 % completion with four times the budget; training variability is unmeasured.
- exp02d: the highest observed completion among the from-scratch PPO checkpoints: 8.7 points more completions than exp02 for about 9 more moves per game, and 96.0 % when sampling instead of taking the argmax (`summary.json`).
- exp03a: eats about 45 cells and then traps itself; its best single game reached 74 of 81 cells (`data/exp03a_ppo_3x4_nocur/episodes.json`, write-up).
- exp03b: triggered the logged 90 % gate only twice in 30M moves, logged frontier 80 -> 78 (`data/exp03b_ppo_3x4_backplay/run.log`; the odd board clamps the effective start at 79), and the saved outcomes do not isolate the corner detour as the failure mechanism (write-up).
- exp03c: the relaxed gate moved the logged frontier from 80 to 76 once (`data/exp03c_ppo_3x4_backplay_relaxed/run.log`) and never again; the selected checkpoint has no observed length-1 completions, which does not isolate an absence of transfer (write-up).
- exp04: the frontier went from 255 to 199 in 7 advances (`data/exp04_ppo_4x4/run.log`); the curriculum-start success then fell below one win per thousand games from about 73M moves and to exactly zero from 77M (`progress.csv`). The write-up treats schedule decay as an untested hypothesis and documents gates contaminated by stale episodes. From length 1 the checkpoint reaches an average length of about 103 cells (fill 0.401), with no observed completions.
- exp05: training-minibatch expert-action accuracy 1.000 after cloning (`data/exp05_bc_4x4/run.log`); aggregate deterministic step counts match the route follower, without proving a head-only representation (write-up).
- exp05b: kept 100 % observed deterministic completion, with nonzero policy updates (`approx_kl` below 2e-5 at every update, `data/exp05b_ppo_4x4_from_bc/progress.csv`). Only one training episode completed, a failure; insufficient trajectory coverage and exploration are possible confounders, not established causes (write-up).

Efficiency beside baselines

Same cell format. For the networks with no sampled completions, these ratios describe only the part of the game they survived, the emptier part, so they flatter those networks against the loop, which is measured over whole

games.

run	board	steps per food	x geometric floor	non-eating moves	won within 4C moves
exp02	2^4	**2.31** (loop 4.39, better; random 9.82, better)	**1.16x** (loop 2.20x, better; random 4.91x, better)	**56.8 %** (loop 77.2 %, better; random 89.8 %, better)	**87.7 %** (loop 45.3 %, better; random 0.7 %, better)
exp02b	2^4	**2.32** (loop 4.39, better; random 9.82, better)	**1.16x** (loop 2.20x, better; random 4.91x, better)	**56.8 %** (loop 77.2 %, better; random 89.8 %, better)	**85.0 %** (loop 45.3 %, better; random 0.7 %, better)
exp02c	2^4	**2.38** (loop 4.39, better; random 9.82, better)	**1.19x** (loop 2.20x, better; random 4.91x, better)	**58.0 %** (loop 77.2 %, better; random 89.8 %, better)	**93.3 %** (loop 45.3 %, better; random 0.7 %, better)
exp02d	2^4	**2.89** (loop 4.39, better; random 9.82, better)	**1.45x** (loop 2.20x, better; random 4.91x, better)	**65.4 %** (loop 77.2 %, better; random 89.8 %, better)	**96.3 %** (loop 45.3 %, better; random 0.7 %, better)
exp03a	3^4	**4.07** (loop 23.43, better; random 73.98, better)	**1.14x** (loop 6.59x, better; random 20.81x, better)	**75.4 %** (loop 95.7 %, better; random 98.6 %, better)	**0.0 %** (loop 0.0 %, tie; random 0.0 %, tie)
exp03b	3^4	**4.11** (loop 23.43, better; random 73.98, better)	**1.16x** (loop 6.59x, better; random 20.81x, better)	**75.7 %** (loop 95.7 %, better; random 98.6 %, better)	**0.0 %** (loop 0.0 %, tie; random 0.0 %, tie)
exp03c	3^4	**4.18** (loop 23.43, better; random 73.98, better)	**1.18x** (loop 6.59x, better; random 20.81x, better)	**76.1 %** (loop 95.7 %, better; random 98.6 %, better)	**0.0 %** (loop 0.0 %, tie; random 0.0 %, tie)
exp04	4^4	**5.57** (loop 64.50, better; random 248.65, better)	**1.11x** (loop 12.90x, better; random 49.73x, better)	**82.0 %** (loop 98.4 %, better; random 99.6 %, better)	**0.0 %** (loop 0.0 %, tie; random 0.0 %, tie)
exp05	4^4	**64.50** (loop 64.50, tie; random 248.65, better)	**12.90x** (loop 12.90x, tie; random 49.73x, better)	**98.4 %** (loop 98.4 %, tie; random 99.6 %, better)	**0.0 %** (loop 0.0 %, tie; random 0.0 %, tie)
exp05b	4^4	**64.37** (loop 64.50, better; random 248.65, better)	**12.87x** (loop 12.90x, better; random 49.73x, better)	**98.4 %** (loop 98.4 %, tie; random 99.6 %, better)	**0.0 %** (loop 0.0 %, tie; random 0.0 %, tie)

What the comparisons say:

- On 16 cells every network beats random on every measure and beats the loop on every efficiency measure, but none reaches the loop's 100 % completion. The strict-curriculum agent comes closest, 3.7 points short, while finishing games in 34 % fewer moves than the loop.
- On 81 and 256 cells the from-scratch networks tie random on completion at 0 %. They beat random on fill by 2.4 to 2.5 times and by 5.2 times, and they beat the loop on steps per food, floor multiple and non-eating share by a wide margin, but only over the games they survived, which end with the board 40 to 57 % full; won within 4C moves is a tie at 0 % for every policy there.
- The clone ties the loop on every reported measure at the printed precision, which is consistent with imitation but does not prove identical internal computation. It exceeds random by 100 points of observed completion, and fine-tuning changed its step count by 0.2 %; a statistically supported efficiency improvement has not been established.
- The gap nobody fills: on 4^4 the network that walks efficiently, 5.57 moves per meal at 1.11 times the geometric reference, has no sampled completions, and the clone completing every sampled deterministic game walks 64.5 moves per meal, 12.9 times that reference. Multiplying 255 foods x 5.00 moves gives about 1,275 moves, but this is illustrative arithmetic, not a proven attainable completion time: food locations and legal paths change with the snake's body.

Experiment 00 - Throughput benchmark (which environment/device configuration to train with)

Date: 2026-09-05 (re-measured with the minibatch sweep built into `snake4d bench`; the first grid of 2026-09-04, not archived, put the batched env first by the same wide margin). Command: `uv run snake4d bench --set batch_size=2048 --set run_name=exp00_benchmark` (run `20260905-010410_bench_exp00_benchmark`). Every number below is read from reports/data/exp00_benchmark/benchmark.json; the table docs/benchmark.md is generated from the same file.

Question

On this laptop (16 logical CPUs, torch using 8 threads, NVIDIA GeForce RTX 5070 Laptop GPU with 8.0 GB, torch 2.14.0+cu130; versions.json), which of the candidate vectorisation schemes gives the most MaskablePPO training steps per second on the 4^4 board, and what budgets follow?

Hypothesis

The research brief predicted, but could not cite a measurement for, the ordering batched single-process env >> DummyVecEnv >> SubprocVecEnv for a cheap environment like snake: per-env Python overhead dominates single envs and inter-process communication dominates subprocesses (SB3 docs on `SubprocVecEnv`, https://stable-baselines3.readthedocs.io/en/master/guide/vec_envs.html; Gymnasium paper Fig. 3 in v4 (Fig. 1 in v2), https://arxiv.org/abs/2407.17032; openai/baselines#608, https://github.com/openai/baselines/issues/608).

Setup

- Board 4^4 (256 cells, observation 1026 floats), MLP [512, 512], `n_epochs=4`, `batch_size=2048` for the grid.
- Grid rows: batched `SnakeVecEnv` with 256 / 1024 / 4096 envs (`n_steps=64`); `DummyVecEnv(16)` and `SubprocVecEnv(16)` over single `SnakeEnv`s (`n_steps=512`); each on cpu and cuda, with `torch.set_num_threads` 1 and 8; about 200k-262k timesteps per row.
- Minibatch sweep: the best grid row re-run with minibatch 2048 / 8192 / 16384.
- fps = `model.num_timesteps / wall time of learn()` (SB3's own `time/fps` is cumulative).
- Env-only rows step the batched env with random legal actions and no policy.

Results

configuration	fps	notes
batched 4096 envs, cuda, batch 2048	20,433	best row of the grid
batched 1024 envs, cuda, batch 2048	18,843	
batched 256 envs, cuda, batch 2048	14,577	
batched 4096 envs, cpu, 8 threads	5,954	1 thread: 1,612
DummyVecEnv(16), best of cpu/cuda	2,006	
SubprocVecEnv(16), best of cpu/cuda	1,881	
env only, batched 256 / 1024 / 4096	234,823 / 136,074 / 126,304 steps/s	no policy

Minibatch sweep on the best row: batch 2048 -> 8192 -> 16384 gives 18,735 -> 30,686 -> 32,967 fps (best measured throughput 32,967 fps). The batch-2048 setting measured 20,433 fps in the grid and 18,735 in the sweep, a difference of about 8 % of the first measurement. Two measurements do not estimate a reliable run-to-run variance.

Analysis and learnings

- The measured ordering is consistent with the hypothesis: the batched env trains 10x faster than the better of the two SB3 vectorisers in this grid. IPC overhead is a plausible explanation for `SubprocVecEnv`, but no profiler or matched vector-count ablation isolates its contribution.
- The environment is not the bottleneck: it steps at 126k-235k steps/s (4.3-7.9 microseconds per env-step) but PPO reaches 20k at batch 2048. This suggests substantial policy/update overhead: 4 epochs x 128 minibatches = 512 gradient steps per 262,144-sample rollout at batch 2048, each with possible fixed overhead; batch 8192 (128 gradient steps) gives 31k fps and 16384 (64 gradient steps) 33k fps with

diminishing returns, so 8192 was adopted.
- Env-only throughput falls as the batch grows (235k at 256 envs to 126k at 4096). Rebuilding the observation tensor (n_envs x 1026 float32, 16 MB at 4096) is a candidate cost; this benchmark does not isolate memory bandwidth from game rules or other work.
- CPU with one torch thread is slow (1.6k fps) because the 1026 -> 512 -> 512 forward pass and the update run single-threaded; 8 threads help 3.7x, the GPU another 3.4x. Training on CPU is a fallback only.
- Larger env batches have higher measured throughput on cuda (15k -> 20k from 256 to 4096 envs). Amortising per-step costs is a plausible explanation, not a measured attribution.

Decisions

- Train on the batched env with `SNAKE_N_ENVS=4096`, `SNAKE_BATCH_SIZE=8192`, `SNAKE_DEVICE=auto` (cuda), `SNAKE_TORCH_THREADS=8`; these are the defaults in `.env.example` and `config.py`.
- Budget arithmetic at ~31k fps: 10M steps = 5 min, 100M steps = 54 min (the real 100M-step run of exp04 took 54 min at 31.1k fps with its evaluations included).
- Evaluation cadence: `SNAKE_EVAL_EVERY=2,097,152` (8 rollouts of 262,144, about a minute) by default; the generated recommendation in docs/benchmark.md targets one evaluation per ten minutes instead and is deliberately not adopted, because the small boards finish long before that.

Experiment 01 - Scripted baselines: is every board completable, and at what step cost?

Date: 2026-09-04. Command: `uv run snake4d evaluate --set size=<2|3|4> --set ndim=4 --set policy=<route|random> --set run_name=exp01_<policy>_<size>x4` (runs `*_evaluate_exp01_*`; per-episode CSVs and summaries in `reports/data/exp01_*`).

Question

Can a scripted policy fill the 2^4, 3^4 and 4^4 boards from a random length-1 start under the episode caps used for training (starvation after more than `4 * cells` consecutive idle steps, `cells^2` total), and how many steps does it need? What does masked random play achieve? These policies supply a scripted completion reference and an uninformed-action reference. Their empirical scores are not universal upper or lower performance bounds.

Hypotheses

- H1: following a Hamiltonian cycle completes every even board (4^4 = 256 cells, 2^4 = 16) in 100 % of episodes, in about `cells^2 / 2` steps (the head must circle the board once per food on average).
- H2: the odd board 3^4 (81 cells, colour classes 41/40, no Hamiltonian cycle) is also completable by a scripted policy, but a plain Hamiltonian *path* follower is not enough.
- H3: masked random play completes some sampled 16-cell episodes and no sampled larger-board episodes.

Setup

100 episodes x 3 seeds per policy and board, a single `deterministic=True` pass (the random baseline still samples its actions), the evaluation protocol of docs/evaluation.md (masked `evaluate_policy`, one batched env per episode, explicit seeds). These are finite samples; see evaluation uncertainty for the interpretation of zero observed failures or successes. The hypotheses are reported retrospectively; independent preregistration is not established (audit).

Results

board	policy	success (mean +- std over seeds)	fill	mean steps to complete	win within `C^2/2` steps
2^4 (16)	route	1.000 +- 0.000	1.000	65.9	100 %
2^4 (16)	random	0.293 +- 0.048	0.816	129.4	15 %
3^4 (81)	route	1.000 +- 0.000	1.000	1,874.5	100 %
3^4 (81)	random	0.000 +- 0.000	0.226	-	0 %
4^4 (256)	route	1.000 +- 0.000	1.000	16,447.8	100 %
4^4 (256)	random	0.000 +- 0.000	0.077	-	0 %

Analysis and learnings

- H1's completion prediction held in every sampled even-board episode; its cost estimate was too high. The observed step cost is close to `C^2 / 4` (16,448 for C = 256), half the `C^2 / 2` worst case, because the food is on average half a lap ahead of the head. This is the efficiency bar: a learned agent should complete 4^4 in far fewer than ~16k steps.
- H2's sampled completion prediction held after a fix. The first version of the route follower used the reflected Gray-code *path* on odd boards and reached only about 4 % fill on 3^4 (run not archived; that follower was removed in PR #6, commit f3b68d7) - worse than random play at 23 %: an open path has a dead end, so the head reaches the end, has to leave the route, and then keeps colliding with its own body near the end. The fix (`hamilton.cycle_minus_corner`) builds a Hamiltonian cycle over the 80 cells other than the corner - an explicit 2D construction plus a "ladder" splice of the corner's column into a neighbouring column for each extra dimension - and the policy enters the corner only when the food is there and at most as many cycle cells would be skipped as are currently free (otherwise the head could overtake its tail). With that, 3^4 is completed in 100 % of sampled episodes with zero recorded fallbacks, in 1,874.5 steps on average. The stale Gray-path run was removed from `runs/`; its numerical and mechanistic account above cannot be independently checked from archived artifacts.

- H3's sample prediction held: masked random play finishes the 16-cell board in 29 % of episodes, with no observed 81- or 256-cell completions; it fills 23 % and 8 % before dying or starving. This does not establish zero population completion probability or isolate the benefit of masking, because there is no unmasked control.
- The starvation cap (`4 * cells`) is never binding for the route follower: the longest wait for food is one lap (`C` steps) plus the detour on odd boards.

Decisions

- The route follower is the demonstration for the Backplay curriculum on every board size, including odd ones (now a cycle segment, not a path prefix).
- Report every learned agent's steps-to-complete against 65.9 / 1,874.5 / 16,447.8 (`steps_to_complete_mean` in `reports/data/exp01_route_*/summary.json`).

Experiment 02 - MaskablePPO on the 2^4 board (16 cells): plain PPO vs curriculum variants

Date: 2026-09-04. Arms (each an `experiments/*.env` file, trained with `uv run snake4d train`, best checkpoint re-evaluated with `uv run snake4d evaluate --set model_path=...`):

arm	curriculum	budget	wall
exp02 `exp02_ppo_2x4`	off	5M steps	2.2 min
exp02b `exp02b_ppo_2x4_backplay`	Backplay, `rho=0.2`, window 8, min 200 episodes	5M	2.3 min
exp02c `exp02c_ppo_2x4_long`	off	20M	8.6 min
exp02d `exp02d_ppo_2x4_backplay_strict`	Backplay, `rho=0.9`, window 4, min 500 episodes	5M	2.1 min

Common settings: 1024 batched envs, `n_steps=32`, `batch_size=4096`, 4 epochs, MLP [512, 512], gamma 0.99, lr 3e-4 -> 1e-5, clip 0.2 -> 0.05, `ent_coef` 0.01, evaluation of 100 true-start episodes every 327,680 steps (655,360 for exp02c), CUDA (36-39k steps/s, final `time/fps`).

Question

Does MaskablePPO alone learn to *complete* the smallest 4D board, how fast, and does the Backplay reverse curriculum change the outcome?

Hypotheses

These are the questions recorded in this retrospective report; independent preregistration is not established. Each arm has one training seed (0), and the three evaluation seeds measure variation for its selected checkpoint. See the scientific audit and evaluation uncertainty for the limits on inference.

- H1: plain PPO reaches 100 % completion within a few hundred thousand steps (the board is tiny).
- H2: the curriculum is unnecessary on 16 cells (exp02b ~ exp02).

Results (best checkpoint, 100 episodes x 3 seeds)

arm	deterministic success	stochastic success	steps to complete	best in-training eval (at step)
exp02 plain 5M	0.877 +- 0.041	0.853 +- 0.034	34.4	0.91 (2.9M)
exp02b Backplay rho 0.2	0.850 +- 0.029	0.843 +- 0.026	34.6	0.91 (3.6M)
exp02c plain 20M	0.933 +- 0.005	0.863 +- 0.025	35.7	0.96 (17.0M)
exp02d Backplay rho 0.9	**0.963 +- 0.009**	**0.960 +- 0.033**	43.5	**1.00 (3.6M)**
route follower (exp01)	1.000	-	65.9	-
random legal (exp01)	0.293	-	129.4	-

Figures: `reports/figures/exp02*_curves.png` (fill, eval success, frontier, episode length) and `exp02*_fill_hist.png`.

Analysis and learnings

- H1 was **not met by these runs**. Plain PPO climbs quickly (eval success 0.41 after 0.3M steps, 0.8 after 1.3M) but plateaus around 0.85-0.93; 4x more steps only lifts the best checkpoint from 0.88 to 0.93. Rollout fill is 0.98, so the agent almost always reaches 15/16 cells.
- Failure mode: of 61 failed evaluation episodes of the 20M model, 35 end at fill 15/16 by a collision (median length 33 for those 35, 31 over all 61 failures; far below the 64-step starvation cap; `eval_episodes.csv`), i.e. the snake boxes itself in with one cell left. Completing the last cell requires the body to be a Hamiltonian path whose head can still reach the free cell. The failure records identify this outcome, but do not establish which representation or reward property caused it.
- H2 is unresolved at the algorithm level. With the loose gate (`rho=0.2`) the frontier ran from 15 to 1 in about 15 seconds of training. The strict configuration produced the highest observed completion among these checkpoints (0.963 deterministic, 0.960 stochastic, an in-training evaluation of 1.00 at 3.6M steps)

at the same 5M budget. This is not a threshold-only ablation: threshold, window and minimum episode count all changed, and neither arm was replicated across training seeds. The callback also pooled completions from earlier frontiers, so neither rapid advancement nor clearing a gate establishes current-stage mastery (audit evidence). The strict configuration became the default (`SNAKE_CURRICULUM_RHO=0.9`, `WINDOW=4`, `MIN_EPS=500`).
- The observed deterministic completion rate exceeds the sampling rate in every arm, with a smaller difference in exp02d. These are descriptive checkpoint comparisons; they do not establish an endgame-confidence mechanism or statistical equivalence.
- Steps to complete (34.4-43.5) are a third to a half below the route follower's 65.9: the learned agents cut across the board instead of following the full cycle. These means condition on successful episodes, while the route follower completed every sampled episode; failure costs matter too.
- The eval curve is noisy at 100 episodes (about +-3 percentage points, the binomial standard error at $p = 0.9$ and $n = 100$ - an estimate, not a measurement), which is why the reported numbers use the best checkpoint re-evaluated with 300 episodes; the in-training "1.00" of exp02d was one such 100-episode sample.

Decisions

- Curriculum defaults switched to the strict gate for exp03b (3^4) and exp04 (4^4).
- Open hypotheses carried forward: closing the last few percent on tiny boards might involve longer training with the strict gate, a lower final entropy, or an endgame-aware feature (reachability of the free region); tested next on the harder boards first.

Experiment 03 - MaskablePPO on the 3^4 board (81 cells, odd parity)

Date: 2026-09-04. Arms (30M steps each, 2048 batched envs, `n_steps=64`, batch 8192, ~10 min each on CUDA):

arm	curriculum	frontier reached	wall
exp03a `exp03a_ppo_3x4_nocur`	off	-	9.9 min
exp03b `exp03b_ppo_3x4_backplay`	Backplay, strict gate `rho=0.9`, window 4, step 1	80 -> 78	9.7 min
exp03c `exp03c_ppo_3x4_backplay_relaxed`	Backplay, `rho=0.8`, window 8, step 4	80 -> 76	10.6 min

The route on this board is the 80-cell cycle minus the corner (docs/game_rules.md), so the frontier starts at 79 effective (the callback logged 80 until the read-back fix in the review PR).

Question

Does PPO complete an odd board, where the last cell must be reached through the corner detour and the final body must be a Hamiltonian path with both ends in the majority colour class? Does the reverse curriculum that worked on 2^4 carry over?

Hypotheses

These hypotheses are reported retrospectively, without established preregistration. Each arm uses training seed 0; three evaluation seeds do not replicate training. Interpretation and provenance limits are recorded in the scientific audit.

- H1: without a curriculum, fill plateaus well below 1 with no completions (parity dead-ends).
- H2: the strict-gate Backplay curriculum reaches the true start and yields completions.

Results (best checkpoint, 100 episodes x 3 seeds, deterministic)

arm	success	fill (eval)	true-start fill during training (mean / best episode)	curriculum-start success
exp03a plain	0.000	0.566	0.5395 / 0.914 (74/81)	-
exp03b strict gate	0.000	0.549	0.5289 / 0.877 (71/81)	0.78 mean; logged gate triggered twice (0.90 -> 79 at 14.9M, 0.91 -> 78 at 15.1M), peak 0.907
exp03c relaxed gate	0.000	0.542	0.5280 / 0.877 (71/81)	0.60
route follower (exp01)	1.000	1.000	-	-

Figures: `reports/figures/exp03*_curves.png`, `exp03*_fill_hist.png`.

Analysis and learnings

- The observed outcome is consistent with H1: every arm fills 54-57 % of the board on average and at most 74/81 cells (exp03a, 161,535 true-start episodes) or 71/81 (exp03b/c, ~80k each; `episodes.json`); no completion. Episodes end by collision at fill ~0.5-0.6, long before the starvation cap. This does not isolate parity as the cause or establish a zero population completion probability (uncertainty).
- H2 was not met in these runs: the curriculum never left the endgame. Starting as a 71-79-cell cycle segment (the frontier is clamped to `route_len - 1 = 79`; the callback logged 80 before the read-back fix; window 4 for exp03b, 8 for exp03c), the curriculum-episode success averaged 78 % (exp03b) and 60 % (exp03c) (`episodes.json`; logged per-rollout values 0.22-0.91 and 0.14-0.80, `progress.csv`) and triggered the 90 % gate only twice (frontier 80 -> 78, then a plateau around 0.83); relaxing the gate to 80 % with larger steps moved the frontier only to 76 and lowered the curriculum success to 60 %. Gate samples mixed episode starts from different frontiers; these crossings therefore do not demonstrate mastery of the stated frontier. The scripted route needs a timed corner detour, but no policy intervention

here identifies failure to learn that detour as the cause.
- The observed true-start training fill is similar across the arms (0.53-0.55), but this does not establish an absence of transfer. The configured 20 % is a probability per reset, not a fraction of transitions: true-start episodes account for **53.93 %** of completed-episode transitions in exp03b and **52.15 %** in exp03c, because their durations differ. These shares exclude unfinished episodes and are reconstructed from raw monitors in the audit evidence.
- Odd parity changes the route construction, but board size, training exposure and curriculum implementation also differ from the small-board runs. These observations do not isolate a causal effect of parity or establish algorithm-level failure.

Decisions

- Treat 3^4 as the parity stress test, not a stepping stone: 4^4 (even, exp04) is the target.
- Next steps if the odd board is pursued: warm-start the policy by imitating the route follower (behaviour cloning on states from route rollouts), then fine-tune with PPO; or add an explicit observation feature for the corner detour condition.

Experiment 04 - MaskablePPO with Backplay on the 4^4 board (256 cells): the headline run

Date: 2026-09-04. `experiments/exp04_ppo_4x4.env`: 4096 batched envs, `n_steps=64`, batch 8192, 100M steps in 54 minutes on CUDA (31.1k steps/s, the final cumulative `time/fps`); Backplay along the 256-cell Hamiltonian cycle with `rho=0.8`, window 16, frontier step 8, 500 episodes per gate decision (relaxed from the 2^4 defaults after exp03b stalled at a 90 % gate).

Question

Can model-free MaskablePPO learn to fill the 4D board from length 1 within a 100M-step budget when episodes are seeded along the Hamiltonian cycle and a success gate moves the start backwards?

Hypothesis

The frontier moves from 255 to 1 within the budget (32 advances of 8 cells at the 80 % gate, each needing at least 500 curriculum episodes), and the true-start completion rate becomes positive once the frontier passes roughly half the board.

This is the hypothesis stated in this retrospective report, not an independently verified preregistration. There is one trained checkpoint lineage (training seed 0); evaluation seeds measure its conditional performance, not training robustness. See the scientific audit for source-provenance and uncertainty limits.

Results

metric	value
curriculum frontier	255 -> 247 (18.6M steps, 9.1 min) -> 239 (22.3M) -> 231 (22.5M) -> 223 (40.6M) -> 215 (46.4M) -> 207 (46.9M) -> 199 (53.7M, 27.1 min); no advance in the remaining 46.4M steps (`run.log`, `progress.csv`)
logged curriculum-start success (frontier logged as 199; gate mixes starts)	0.81 at the advance (53.7M), 0.60 at 56.1M, below 0.001 from ~73M, exactly zero from ~77M to 100M steps
rollout fill (all starts)	0.81 at 26M, falling to 0.71 at 100M
true-start fill (training)	mean 0.377, best episode 0.660 (169/256 cells); 81,995 true-start episodes of 624 steps on average (`episodes.json`)
true-start completion (eval, 47 evaluations of 100 episodes)	0.000 throughout
best checkpoint, 100 episodes x 3 seeds, deterministic / stochastic	success 0.000 / 0.000, fill 0.401 / 0.395
route follower (exp01)	success 1.000, 16,448 steps

Figures: `reports/figures/exp04_ppo_4x4_curves.png`, `exp04_ppo_4x4_fill_hist.png`.

Analysis and learnings

- **Negative result for this run.** No recorded true-start episode completed the 4^4 board; the best true-start training episode filled two thirds of it. The route follower demonstrates feasibility, but these observations do not show that PPO cannot learn a completer.
- **The curriculum collapsed instead of advancing.** Seven advances happened between 18.6M and 53.7M steps (gate success 0.80-0.84 each time), then the frontier stayed at 199 and the success rate for curriculum starts (57-73 free cells at that frontier) fell to zero in the last quarter of training. The learning rate and clip range decay over the 100M budget regardless of curriculum progress, but no constant-schedule control isolates that choice as the cause. Schedule decay is a hypothesis.
- **Advances did not establish stage mastery.** The callback counted episodes begun before a frontier change toward the next gate. Excluding definitely stale episodes changes the 239 -> 231 gate from 740/892 to 394/537 successes (73.37 %), and the 215 -> 207 gate from 799/963 to 355/492 (72.15 %). Both are below the required 80 %, and the latter also has fewer than 500 episodes. These are filtered retrospective counts, not evaluations of a fixed current stage. The audit records their derivation and the resulting alternative explanation for curriculum failure.
- **Transfer is unresolved.** Curriculum-start fill (0.88, `episodes.json`) and true-start fill (0.38-0.40) have

different initial lengths, so their difference is not a measure of transfer. There is no matched no-curriculum 4^4 control or state-distribution ablation. The configured true-start probability is 20 % per reset, not 20 % of transitions.

- **What the agent did learn**: from length 1 it eats about 100 cells (eval fill 0.40) before colliding - 5.2x the random floor (0.077) - in episodes of roughly 570-630 steps (evaluation 566, training 624). This describes observed food collection; it does not identify an internal greedy strategy.

Decisions and next steps

These are proposals for future experiments; no new training or policy evaluation was performed for the audit corrections.

1. First associate every episode with its reset frontier so stale episodes cannot establish current-stage mastery. Then compare schedule variants with the gate held fixed and independent training replications; this separates implementation repair from the schedule hypothesis.
2. Warm-start by imitation: behaviour-clone the policy on (observation, action) pairs from the route follower (which is available in unlimited quantity from the batched env), then fine-tune with MaskablePPO - the network then starts from a policy that already completes the board and RL can then be tested for an efficiency improvement. Exp05 reports the resulting observation; it does not isolate the causal reason for the difference from exp04.
3. Smaller frontier steps (4) with a longer budget (300M+ steps, ~3 h), and evaluation of the curriculum-start success on a fixed set of frontier states instead of the moving window.
4. As a fallback, search at decision time (AlphaSnake-style MCTS with the learned value function).

Experiment 05 - Imitation warm start on 4^4: behaviour cloning, then PPO fine-tuning

Date: 2026-09-04. Arms: `experiments/exp05_bc_4x4.env` (`snake4d imitate`) and `experiments/exp05b_ppo_4x4_from_bc.env` (`snake4d train --set model_path=<exp05>/bc_model.zip`).

Question

exp04 recorded no true-start completions in its 100M-step run, while the scripted route follower completed every sampled episode. Can the *same* policy network (MLP [512, 512] on the `4C + 2` observation) learn to complete the board by imitating the route follower, and does PPO fine-tuning then keep the completion rate while reducing the ~16k steps per game?

Hypotheses

These are retrospective hypotheses; independent preregistration is not established. Both arms use training seed 0. The three evaluation seeds measure each fixed checkpoint, not variation across independent training runs (scientific audit).

- H1: the network can represent the route follower on 4^4 (the action is a function of the head cell only, a 256-entry lookup) and behaviour cloning reaches near-100 % expert-action accuracy.
- H2: the cloned policy completes the board from the true start in 100 % of deterministic evaluations; sampled actions occasionally leave the cycle and lower the rate.
- H3: PPO fine-tuning with gentle settings (lr 1e-4 -> 1e-5, clip 0.1 -> 0.05, no entropy bonus, no curriculum) keeps completion near 100 % and shortens games by taking shortcuts.

Setup

- Data: 4096 batched envs x 64 steps = 262,144 (observation, expert action, mask) samples. The reset mixture uses length-1 starts with probability 0.2 and curriculum starts sampled across cycle lengths; subsequent expert transitions are not uniformly distributed over lengths. Expert = `RoutePolicy` (Hamiltonian cycle follower); there is no held-out action dataset.
- Cloning: negative log-likelihood of the expert action under the masked policy distribution, Adam lr 1e-3, minibatch 8192, 20 epochs; the value head is untouched. Wall time about 12 s for the model build plus the 20 epochs, about 17 s including expert-data collection (`run.log`).
- Evaluation: docs/evaluation.md protocol, 100 episodes x 3 seeds, deterministic and stochastic.

Results

policy	deterministic success	stochastic success	fill (det / stoch)	steps to complete (det)
behaviour-cloned network (exp05)	**1.000 +- 0.000**	0.787 +- 0.041	1.000 / 0.901	16,448
clone + PPO fine-tuning, best checkpoint (exp05b)	**1.000 +- 0.000**	0.763 +- 0.017	1.000 / 0.884	16,414
route follower (exp01, scripted)	1.000	-	1.000	16,448
PPO + Backplay from scratch (exp04, 100M steps)	0.000	0.000	0.401 / 0.395	-

Training-minibatch expert-action accuracy after cloning: 1.000 (loss 2e-4 after 20 epochs). This logged accuracy aggregates minibatch accuracies during the final training epoch, each measured immediately after its update. It is neither held-out generalization accuracy nor a fixed-final-model dataset evaluation (imitation.py).

PPO fine-tuning (exp05b): 20M steps, 22.5 min, from `bc_model.zip`

metric	value
in-training evaluations (100 true-start episodes every 2.1M steps)	success 1.00 at every one of the 9 evaluations
mean episode length at those evaluations	16,218 - 16,514 steps (`evaluations.npz`)
`train/approx_kl`	7.9e-7 to 1.7e-5 at every update (prints as 0.000; `progress.csv`)

throughput	15k fps; no profiling ablation isolates the difference from exp04

H3's efficiency component is **unestablished**. Both selected checkpoints completed all 300 deterministic evaluation episodes, but the mean step-count difference alone does not establish an improvement. Nonzero KL and entropy contradict the earlier claims that the policy did not change and every sampled action equalled argmax. Stochastic evaluations recorded **64/300 failures before** and **71/300 after** fine-tuning; absence of an entropy bonus is not zero policy entropy. Exponentiating a mean training loss would give a geometric mean of assigned expert probabilities on those training examples, not a universal action probability at test time.

Training coverage is a substantial confound: 4096 environments each received **4,928 steps**, whereas a successful deterministic evaluation game took roughly 16,414 steps. Across training, only **one completed episode, a failure**, was recorded and there were **zero successful training episodes** (episodes.json). PPO still updates on partial trajectories, but this run supplies little evidence about fine-tuning with complete successful trajectories. The audit links the saved-log derivation; it does not establish insufficient exploration as the cause.

Analysis and learnings

- H1 and H2 are consistent with the observed training accuracy and deterministic evaluations. The cloned checkpoint completed every sampled deterministic episode and matched the route follower's aggregate step counts (16,448 on average, seed by seed). This is consistent with route imitation, but aggregate equality does not prove identical trajectories or a head-only internal representation; that would require interventions on non-head observation features.
- Sampling actions completes 74-84 % across the clone's three seed groups. This establishes a mode-dependent outcome for this checkpoint. Saved aggregates alone do not identify which off-cycle actions led to each failure or guarantee that sharpening would improve the policy.
- Dense expert supervision and varied starting lengths are plausible reasons for BC's outcome. The comparison with exp04 also changes objective, initialization, data and training exposure, and exp04 has a curriculum defect. It does not isolate supervision or establish absence of a train/evaluation state-distribution shift.
- Cost: about twelve seconds for cloning versus 54 minutes for exp04. Expert collection takes additional time as recorded above, and the hand-written route is prior problem knowledge; this is not an equal-information or equal-compute algorithm comparison.

Decisions

- The cloned checkpoints are demonstrated 4^4 completers on the sampled deterministic games; generalization to larger boards and robustness across training seeds remain open.
- A future fine-tuning comparison should first provide sufficient per-environment trajectory coverage, then vary exploration separately while preserving a completion metric and a prespecified analysis. Entropy bonuses or label smoothing are candidate interventions, not remedies demonstrated by exp05b. No new model execution was performed for these corrections.

Independent scientific audit of the archived experiments

Author: Libor Burian. Audit date: 2026-10-22. Code baseline: `cd30fd7`.

Scope, method and conclusion

This is a retrospective audit, not a new training experiment or a preregistration. It independently checks the source, historical Git trees, CSV/JSON/NPZ results and existing local monitor logs; previous audit conclusions are not treated as evidence. No model, training, inference or policy rollout was run. Checkpoint ZIP metadata was read as JSON without deserializing model objects.

The numerical evaluation summaries agree with their episode records. However, the curriculum gate accepts stale episodes, all trained configurations have one training seed (0), and several causal and reproducibility claims exceed the available evidence. The surviving findings describe the particular evaluated checkpoints, boards, episode caps and action-selection modes. They do not establish algorithm-level superiority, training robustness, universal completion or impossibility.

The audit checked 16 evaluation archives: 7,800 episode records, 78 seed/mode groups, 26 pooled groups and 780 aggregate fields. Each seed/mode group has 100 episodes. The count treats each `win_within` dictionary as one field while checking all four entries. Success, final fill, return, episode length, successful-episode length, step-capped wins and evaluation-seed mean/standard deviation all agree. Every recorded success has fill 1; fills are valid cell counts and lengths respect the configured caps. See the standalone checks and the original `reports/data/exp*/eval_episodes.csv` and `summary.json` files; those files are unchanged.

1. Curriculum gates mix episodes from different frontiers

`Backplay._on_step` accepts every finished episode with `start_len > 1`. Advancing clears the result list, but `SnakeBatch.set_curriculum` only changes future resets. Episodes already in flight retain their easier starting states and can enter the next gate. There is no reset-time frontier identifier in the episode records.

Saved exp04 records demonstrate the problem, independently of any proposed explanation for the later collapse:

Frontier change	Transition count	Mixed gate wins/episodes	Definitely stale starts/wins	Remaining wins/episodes
239 -> 231	22,544,384	740/892 = 82.96%	355 / 346	394/537 = 73.37%
215 -> 207	46,923,776	799/963 = 82.97%	471 / 444	355/492 = 72.15%

"Definitely stale" means `start_len` exceeds the frontier being tested. Both remaining rates are below the configured 80% gate; the second has fewer than the required 500 episodes. Remaining episodes can still be stale when old and new length windows overlap. Therefore these exclusions do not recover an uncontaminated current-frontier estimate.

The 1,855 archived curriculum rows preserve their original monitor line numbers. The evidence manifest records the complete source monitor's SHA-256, timestamp origin, extraction windows and gate counts. The mixed rates match exp04 progress exactly. Windows are between consecutive frontier changes in run.log: 21:13:49.303 to 21:13:57.664 and 21:25:55.578 to 21:26:12.089 on 2026-09-04, UTC+02:00. Subtracting the monitor's Unix `t_start` gives the manifest's lower-exclusive, upper-inclusive elapsed-time windows; only `start_len > 1` rows enter the gate.

This defect invalidates the interpretation that advancement demonstrated mastery at each stage. It does not change the independently recorded true-start evaluation scores. Schedule decay and start-distribution mismatch remain hypotheses; neither was isolated by an ablation, and this implementation defect is an additional possible contributor. No causal share is estimated here. Future implementation requirements are specified in the RL design.

2. Uncertainty and the unit of replication

All ten training/imitation `config.json` files specify `seed = 0`. Final evaluation reseeds the same loaded checkpoint with 0, 1 and 2; `success_rate_std` is the population standard deviation of three evaluation-batch

rates, not a confidence interval or variability across trained networks. The strict versus loose curriculum also changes threshold, window and minimum episode count together. Their scores compare two configurations and two particular checkpoints, not an isolated threshold intervention. See exp02.

Illustrative two-sided 95% Wilson intervals for a fixed checkpoint are:

Observed deterministic wins	Example	Conditional completion interval
289/300	exp02d	93.55-97.94%
300/300	exp05 and exp05b, separately	98.74-100.00%
0/300	each exp03/exp04 checkpoint, separately	0.00-1.26%

These use the NIST Wilson formula under an independent Bernoulli-episode interpretation for the fixed checkpoint and evaluation distribution. They do not account for training variance, adaptive experiment selection or multiple comparisons, and are not algorithm-comparison significance tests. More episode resampling cannot recover missing training replications. See Patterson et al., Agarwal et al. and the evaluation protocol.

Training validation uses seed 1000, while final evaluation uses 0, 1 and 2. Shared seed 0 alone does not establish train/test trajectory leakage: the batch sizes and random-call schedules differ. No direct trajectory leak was demonstrated. Conversely, common evaluation seeds do not establish matched episode pairs: food spawning and autoresets consume a shared batch generator according to policy-dependent events, and CSV rows are in completion order without environment IDs. Pairing rows by ordinal position would be unjustified. Reused evaluation results and adaptive configuration choices make these comparisons exploratory.

3. Fine-tuning and behaviour-cloning claims

Exp05b's episode summary contains one completed training episode, a failure, and zero successful episodes. Its 20,185,088 transitions over 4,096 environments give 4,928 steps per environment, compared with about 16,414 steps per successful evaluation game. PPO still learns from partial trajectories and bootstrapped values, but this run did not observe a successful complete training trajectory. Limited trajectory coverage is a major alternative explanation for the lack of an established efficiency improvement.

The progress log has nonzero recorded KL, entropy and policy-gradient terms. Stochastic evaluations contain 64/300 failures for BC and 71/300 for fine-tuning. The deterministic mean successful-game length changes from 16,447.84 to 16,413.5567 steps. These observations contradict "every sampled action equals argmax" and do not support "the policy did not change at all". The small sample-mean speed difference is descriptive; a reliable improvement has not been established.

`imitation.py` logs loss before each minibatch update and accuracy after updating on that same minibatch. Its rounded 1.000 accuracy is online training accuracy, not held-out accuracy of one fixed final checkpoint. Exponentiating the averaged NLL produces a geometric mean over training minibatches/parameter states, not a measured deployment-wide action probability. Matching the route follower's 300 deterministic episode records supports behavioural agreement on that evaluation; it does not identify an internal head-position-only mechanism or prove the absence of distribution shift. The archived 300 deterministic BC wins remain valid observations. See the imitation report.

4. Reset probability is not transition share

The 20% true-start reset probability does not imply a fifth of the training transitions. Longer true-start episodes contribute more steps. Exact sums from the local completed-episode monitors:

Run	True-start transitions	All completed-episode transitions	True-start share
exp03b	16,112,937	29,880,185	53.93%
exp03c	15,582,883	29,883,203	52.15%
exp04	51,186,306	99,301,360	51.55%

Counts and source hashes are in evidence.json. They exclude unfinished episodes at the training cutoff, so they are not exact fractions of the entire rollout buffer. The original archived `episodes.json` files round mean lengths and cannot recover these integer totals exactly. Those rounded summaries corroborate the approximate shares. Comparing final fill across curriculum and true-start episodes also confounds initial fill: curriculum episodes begin with much of the board occupied. These comparisons do not isolate transfer.

5. Preregistration and source provenance

Repository history does not establish preregistration of the executed configurations or analysis:

Evidence, 2026-09-04 UTC + 02:00	Run completed	First corresponding commit
exp02d configuration	20:42:29	`67167f0`, 20:47:43
exp05 configuration and imitation implementation	22:17:46.647	`575430d`, 22:18:16
exp04 executed rho 0.8/window 16/step 8	21:56:36	`d06d4a0`, 22:14:34

These are Git chronology checks against the runs' archived `run.log` files. Earlier exp04 configuration existed with different settings. Unseen external preregistration cannot be ruled out, but is not established by this repository. Describe these as exploratory experiments.

The imitation train/evaluation versions file records `e483fe513be150f7ec5439b68d694e727b6b61dd`; that Git tree lacks `src/snake4d/imitation.py`. The fine-tuning recorded revision also lacks the loading overrides used by the actual run; its checkpoint JSON metadata and progress logs nevertheless agree with the stated learning-rate, clip and entropy settings. This is a source-provenance gap, not evidence of a hyperparameter mismatch. Recording HEAD without dirty-source contents does not identify executed code.

Historical recorded PPO revisions also contain terminal-observation aliasing: `obs[i]` was stored as a view and subsequently overwritten during autoreset. Current code uses `.copy()`. The MaskablePPO timeout bootstrap consumes that observation. Saved rewards identify 26, 25, 23, 23, 1, 0, 1 and 0 truncations in exp02, exp02b, exp02c, exp02d, exp03a, exp03b, exp03c and exp04, respectively. These are retrospective classifications: collision reward is `foods - 1 - .001*(steps - 1)`, truncation reward `foods - .001*steps`; their separation is .999 and the largest observed residual is .00266. Thus the recorded-code defect had limited potential exposure and cannot explain exp04's failure, which has zero detected training truncations. Missing exact source snapshots prevent complete historical attribution.

Reproduce the checks and understand their limits

```
uv run --no-sync pytest tests/test_scientific_audit.py -q
uv run --no-sync pytest tests/test_docs_numbers.py -q \
  -k "not test_mlp_params_matches_the_built_policy"
```

The first command imports no model code. The second selects only artifact, prose and arithmetic checks; it excludes the model-construction test. Do not run `train`, `imitate`, `evaluate`, `bench`, `pipeline`, or the full test suite to reproduce this audit: some existing tests run models.

Clones can recompute the evaluation summaries, intervals, fine-tuning coverage and gate counts from committed artifacts. The three full monitor logs remain ignored; tests skip full-monitor corroboration when they are absent. Their hashes identify the inspected originals but do not substitute for distributing those originals. The committed gate excerpts support the local gate calculation; independently checking extraction completeness requires the full hashed monitor. Git chronology can be inspected with `git show` and `git log --follow` without executing code.

No new experimental outcomes are asserted. Future frontier identifiers, source manifests, checkpoint hashes and replication requirements are specifications in the methodology and architecture, not features implemented by this documentation correction. Existing checkpoints and historical scores remain unchanged; stronger causal and algorithm-level claims require new, separately authorized work.